

Data Studio API usage

TonAI Data Studio exposes a REST API under `/api/v1`. It accepts Hugging Face-compatible repository folders, but it is not a drop-in implementation of the `huggingface_hub` protocol.

The examples below assume the Compose deployment is available at `http://localhost:3000` and that `curl` and `jq` are installed:

bash

```
export DATA_STUDIO_URL=http://localhost:3000
export DATA_STUDIO_API="$DATA_STUDIO_URL/api/v1"
```

For a direct API development process on port 8000, set `DATA_STUDIO_URL=http://localhost:8000`. Interactive OpenAPI documentation is available at `http://localhost:8000/docs`.

Authentication

Public datasets can be read without credentials. Creating or changing a dataset requires either a signed browser session or a personal API token. Private datasets can only be read by their owner or a workspace administrator; internal datasets can be read by any signed-in user.

Create an account or sign in

The following commands keep the signed session in a local cookie jar:

bash

```
curl --fail-with-body --silent --show-error \
  --cookie-jar data-studio.cookies \
  --header 'Content-Type: application/json' \
  --data '{"username": "owner", "password": "replace-with-a-strong-password"}' \
  "$DATA_STUDIO_API/auth/register" | jq
```

For an existing account:

bash

```
curl --fail-with-body --silent --show-error \
  --cookie-jar data-studio.cookies \
  --header 'Content-Type: application/json' \
  --data '{"username": "owner", "password": "replace-with-a-strong-password"}' \
  "$DATA_STUDIO_API/auth/login" | jq
```

The first registered account is the workspace administrator.

Create a personal API token

Token creation requires a signed session. The raw token is returned once, so store it in a secret manager and never commit it to source control.

bash

```
export DATA_STUDIO_TOKEN="$(
  curl --fail-with-body --silent --show-error \
    --cookie data-studio.cookies \
    --header 'Content-Type: application/json' \
    --data '{"name": "local-cli", "scopes": ["read", "write"]}' \
    "$DATA_STUDIO_API/auth/tokens" | jq -er '.token'
)"
```

Use the token on subsequent requests:

bash

```
curl --fail-with-body --silent --show-error \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  "$DATA_STUDIO_API/auth/me" | jq
```

Use only the read scope for clients that do not create, upload, update, or delete datasets.

Create a dataset

bash

```
curl --fail-with-body --silent --show-error \
  --request POST \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  --header 'Content-Type: application/json' \
  --data '{
    "namespace": "owner",
    "slug": "sentiment-demo",
    "visibility": "private",
    "description": "Sentiment examples"
  }' \
  "$DATA_STUDIO_API/datasets" | jq
```

Valid visibility values are private, internal, and public.

Upload a repository folder

Uploading is a three-step transaction: create an upload, send its files and relative paths, then publish it. The API preserves each supplied relative POSIX path.

1. Create an upload

bash

```
export UPLOAD_ID="$(
  curl --fail-with-body --silent --show-error \
    --request POST \
    --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
    --header 'Content-Type: application/json' \
    --data '{"commit_message": "Initial import"}' \
    "$DATA_STUDIO_API/datasets/owner/sentiment-demo/uploads" | jq -er '.id'
)"
```

2. Send files

Every files entry must have a paths entry at the same position. Paths are relative to the root of the repository and must use /, even when the client runs on Windows.

bash

```
curl --fail-with-body --silent --show-error \
  --request POST \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  --form 'files=@README.md;type=text/markdown' \
  --form 'files=@data/train.parquet;type=application/vnd.apache.parquet' \
  --form 'paths=README.md' \
  --form 'paths=data/train.parquet' \
  "$DATA_STUDIO_API/uploads/$UPLOAD_ID/files" | jq
```

Large folders may be sent in multiple requests. Do not send the same repository path twice.

3. Publish the revision

bash

```
export REVISION_ID="$(
  curl --fail-with-body --silent --show-error \
    --request POST \
    --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
    --header 'Content-Type: application/json' \
    --data '{"expected_file_count":2}' \
    "$DATA_STUDIO_API/uploads/$UPLOAD_ID/complete?include_files=false" \
    | tee /tmp/data-studio-revision.json \
    | jq -er '.revision_id'
)"
```

Publishing is content-addressed and idempotent: retrying the same completed repository tree returns the existing immutable revision.

Browse datasets and revisions

List all datasets visible to the current credential:

bash

```
curl --fail-with-body --silent --show-error \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  "$DATA_STUDIO_API/datasets" | jq
```

Read repository metadata and revision history:

bash

```
curl --fail-with-body --silent --show-error \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo" | jq

curl --fail-with-body --silent --show-error \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo/revisions" | jq
```

Page through repository files:

bash

```
curl --fail-with-body --silent --show-error \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo/tree/$REVISION_ID/page?offset=0&limit=100" \
  | jq
```

Query preview rows

First list detected configs and splits:

bash

```
curl --fail-with-body --silent --show-error \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo/configs?revision=$REVISION_ID" | jq
```

Then request a page from a config and split:

bash

```
curl --fail-with-body --silent --show-error --get \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  --data-urlencode "revision=$REVISION_ID" \
  --data-urlencode 'offset=0' \
  --data-urlencode 'limit=50' \
  --data-urlencode 'columns=text,label' \
  --data-urlencode 'filter={"column":"label","op":"eq","value":1}' \
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo/viewer/default/train" | jq
```

Supported filter operations are eq, ne, contains, gt, gte, lt, and lte. Preview rows are a bounded sample; inspect the response capabilities before assuming full-dataset operations.

Download files or a complete revision

Download one source file

bash

```
curl --fail-with-body --location \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  --output train.parquet \
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo/blob/$REVISION_ID/data/train.parquet"
```

Source downloads are streamed and retain the uploaded bytes.

Pull a complete repository

The tree endpoint returns every path in a revision. The following Bash workflow downloads each source object and recreates the original repository layout locally. Set `REVISION_ID` to an immutable revision ID for a reproducible snapshot, or use `main` to pull the latest revision.

bash

```
export DATASET_PATH=owner/sentiment-demo
export REVISION_ID=main
export DESTINATION=sentiment-demo

mkdir -p "$DESTINATION"

curl --fail --silent --show-error \
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
  "$DATA_STUDIO_API/datasets/$DATASET_PATH/tree/$REVISION_ID" \
  | jq -r '.[].path' \
  | while IFS= read -r path; do
    encoded_path="$(jq -nr --arg path "$path" '$path | @uri')
    mkdir -p "$DESTINATION/${dirname "$path"}"
    echo "Downloading $path"
    curl --fail --silent --show-error --location --retry 3 \
      --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \
      --output "$DESTINATION/$path" \
      "$DATA_STUDIO_API/datasets/$DATASET_PATH/blob/$REVISION_ID/$encoded_path"
  done
```

Public datasets do not require credentials; remove both Authorization headers when pulling one. Private datasets require an owner or administrator token, while internal datasets require a token from any signed-in user. The Studio accepts Hugging Face-compatible layouts but is not a drop-in implementation of the Hub download protocol, so `hf download` cannot pull these repositories.

Update or delete a dataset

bash

```
curl --fail-with-body --silent --show-error \  
  --request PATCH \  
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \  
  --header 'Content-Type: application/json' \  
  --data '{"slug":"sentiment-v2","description":"Updated description","visibility":"internal"}' \  
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo" | jq
```

Changing `slug` renames the dataset and changes its URL. Existing immutable revisions and downloads remain available under the new repository URL.

Deletion permanently removes repository metadata and its source and derived object prefixes:

bash

```
curl --fail-with-body --silent --show-error \  
  --request DELETE \  
  --header "Authorization: Bearer $DATA_STUDIO_TOKEN" \  
  "$DATA_STUDIO_API/datasets/owner/sentiment-demo"
```

Errors

API errors use a consistent problem document:

json

```
{  
  "type": "about:blank",  
  "title": "Access denied",  
  "status": 403,  
  "detail": "Only the dataset owner can change this dataset.",  
  "code": "forbidden",  
  "instance": "/api/v1/datasets/owner/sentiment-demo"  
}
```

Use `--fail-with-body` in automation so `curl` returns a non-zero status while retaining the error body. Treat 401 as a missing or expired credential, 403 as insufficient scope or ownership, 409 as a resource conflict, and 422 as invalid input or dataset validation failure.